Documenting C++ with Doxygen and Sphinx - Exhale

Rohit Goswami · 2020-09-22 · documentation · workflow · cpp · sphinx · doxygen

> *This post outlines a basic workflow for C++ projects using Doxygen, Sphinx, and Exhale.*

Background

My project proposal for documenting Symengine was recently accepted for the Google Summer of Docs initiative. In the past I have been more than happy to document C++ code using **only** Doxygen (with pretty fantastic results), while keeping example usage separate (d-SEAMS wiki). Though this is still a feasible method, a monolithic multi-project setup might benefit from Sphinx, which is what will be covered.

Goals

A couple of goals informed this approach:

- We expect our documentation to link to the source files
- We expect a lot of Python developers to contribute
  - Hence Sphinx
- We would like to write `.ipynb` files into the docs
  - Another reason to use Sphinx (via MyST{NB})

Folder Structure

```
tree -d $prj/ -L 2
```

```
.
├── docs
│   ├── Doxygen
│   └── Sphinx
├── nix
│   └── pkgs
├── projects
│   └── symengine
└── scripts

8 directories
```

Essentially we have a `scripts` directory to store basic build scripts, and two kinds of documentation folders.

Basic Doxygen

The `doxygen` setup is beautifully simple:

```
cd docs/Doxygen
doxygen -g
# Easier to edit
mv Doxyfile Doxyfile.cfg
```

Since `Doxyfile` should be updated by subsequent versions of `doxygen`, it is best to separate the project settings. We will therefore modify some basic settings in a separate file

```
touch Doxyfile-prj.cfg
vim Doxyfile-prj.cfg # or whatever
```

Edit the file to be (minimally):

```
@INCLUDE               = "./Doxyfile.cfg"
GENERATE_HTML          = NO
GENERATE_XML           = YES
XML_PROGRAMLISTING = NO

# Project Stuff
PROJECT_NAME           = "myProject"
PROJECT_BRIEF          = "Dev docs"
OUTPUT_DIRECTORY       = "./gen_docs"

# Inputs
INPUT                  = "./../../projects/symengine/symengine"
RECURSIVE              = NO
```

With this we will now be able to obtain the `xml` files for the rest of this setup.

Exhale

For our first attempt, we will focus on the automation of Sphinx using the exhale tool.

```
# Basic setup
poetry init
poetry add exhale breathe
```

Now we can generate the basic Sphinx structure.

```
# Separate source and build
sphinx-quickstart --sep --makefile docs/Sphinx \
    --project "My Proj" \
    --author "Juurj" \
    --release "latest" \
    --language "en"
```

This allows us to generate the Sphinx documentation we require, with some changes to the `docs/Sphinx/source/config.py` file (lifted from the exhale documentation):

```
extensions = [
    'breathe',
    'exhale',
]

# -- Exhale configuration ------------------------------------------------
# Setup the breathe extension
breathe_projects = {
    "My Proj": "./../../Doxygen/gen_docs/xml"
}
breathe_default_project = "My Proj"

 # Setup the exhale extension
exhale_args = {
    # These arguments are required
    "containmentFolder":     "./api",
    "rootFileName":          "library_root.rst",
    "rootFileTitle":         "Library API",
    "doxygenStripFromPath":  "..",
    # Suggested optional arguments
    "createTreeView":        True,
    # TIP: if using the sphinx-bootstrap-theme, you need
    # "treeViewIsBootstrap": True,
}

# Tell sphinx what the primary language being documented is.
primary_domain = 'cpp'

# Tell sphinx what the pygments highlight language should be.
highlight_language = 'cpp'
```

We also need to add the output to the `index.rst` use the following:

```
.. toctree::
   :maxdepth: 2
   :caption: Contents:

   api/library_root
```

At this point we are ready to manually build our documentation.

```
cd docs/Doxygen
doxygen Doxyfile-prj.cfg
cd ../Sphinx
make html
```

This is still pretty cumbersome though. We can view our documentation in a more pleasant manner with `darkhttpd`.

```
darkhttpd docs/Sphinx/build/html
```

With this we now beautify the documentation (with the sphinx_book_theme):

```
poetry add sphinx-book-theme
```

We need to set the theme as well (in the Sphinx `config.py` file):

```
html_theme = 'sphinx_book_theme'
```

This leads to some pretty documentation.

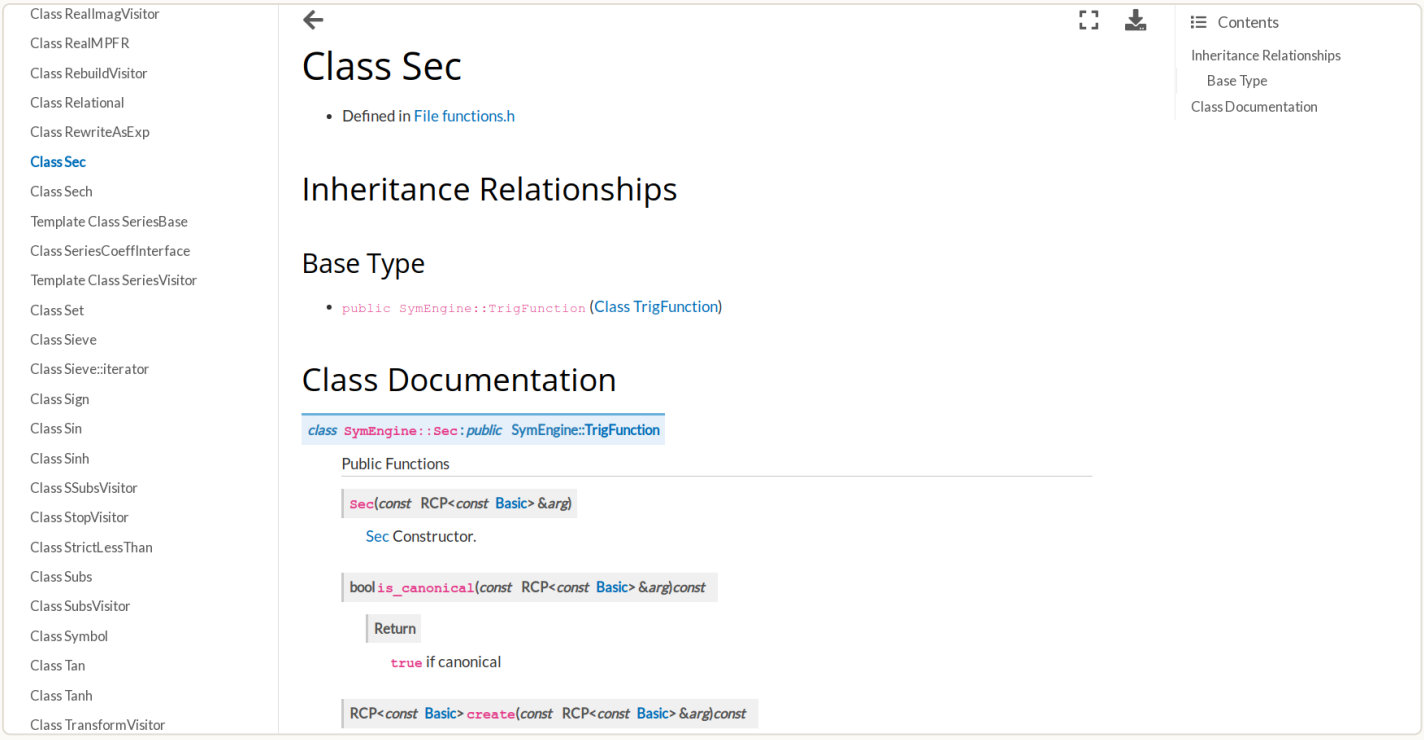

Figure 1: Generated documentation (Exhale)

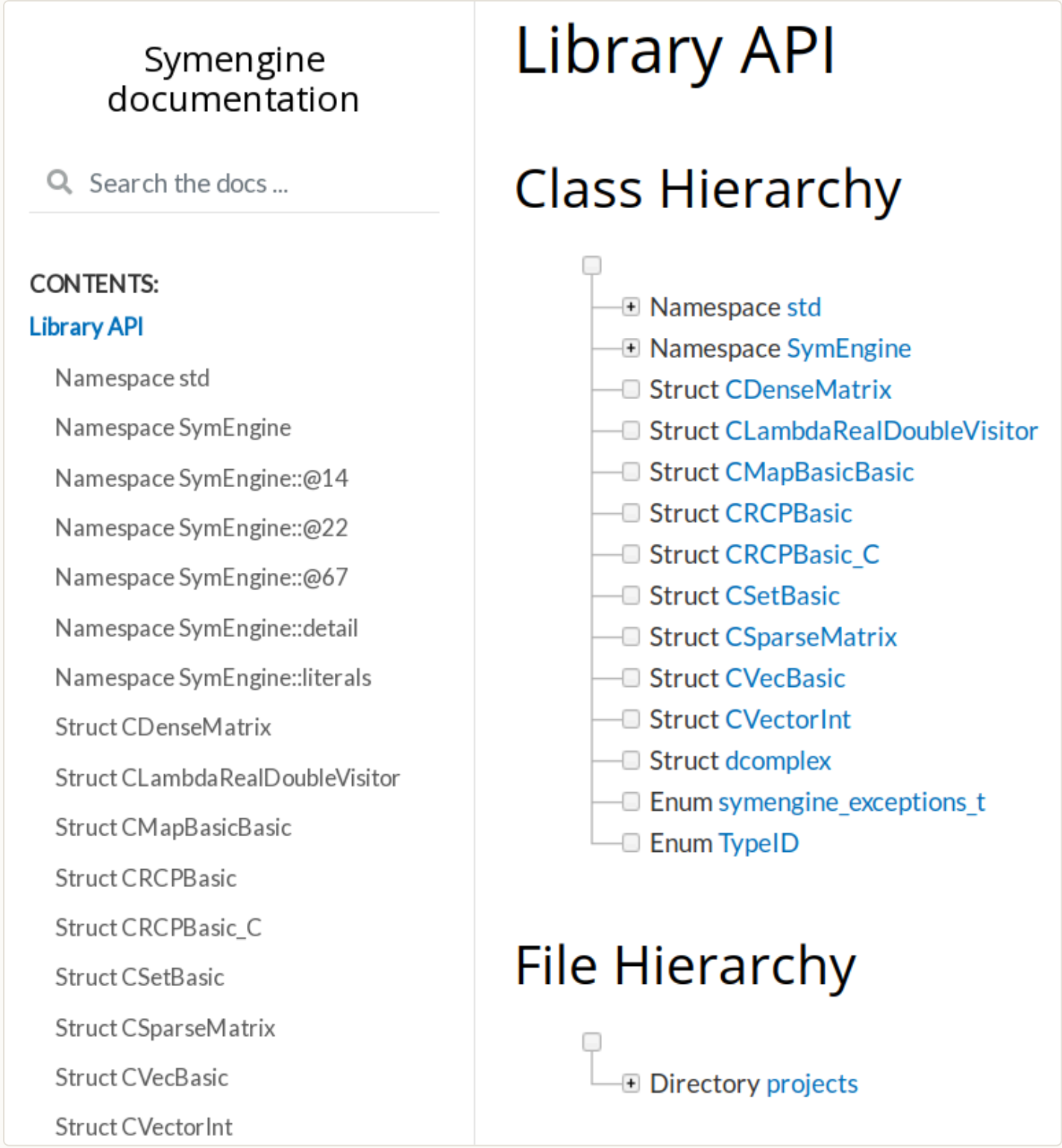

Figure 2: Exhale does a great job with file-based hierarchy.

Conclusions

At this point, we have a basic setup, which we can tweak with a bunch of themes, and/or different parsers, but this is still pretty rough around the edges. However, a caveat of this setup is that the actual contents of the source are not visible in the generated documentation. In the next post, we will look at automating this setup for deploying with Travis.